

Id based Protocols

Motivation: PKI

To use Public Key Cryptography we need to bind identities and keys.

Public Key Infrastructures

A Certification Authority (CA) issues certificates:

$$\left. \begin{array}{l} \bullet U \text{ user's identity} \\ \bullet PK \text{ public key} \\ \bullet D_1 \text{ issue date} \\ \bullet D_2 \text{ expiration date} \end{array} \right\} \begin{array}{l} \text{Certificate}(U, PK) \\ \text{Sig}_{CA}(U, PK, D_1, D_2) \end{array}$$

Certificate Revocation Problem

Motivation: PKI

Before performing the cryptographic operation involving the public key, we must validate $\text{Certificate}(U, PK)$.

Easy for signature schemes. User U sends the certificate along with its signature on a message m

$$(\text{Certificate}(U, PK), \text{Sig}_{PK}(m), m)$$

Difficult for encryption schemes. Before sending a message m to user U , we should know if it is in possession of a valid certificate.

We would like to perform the public operation
without extra communication.

Identity-Based Schemes

- A type of public-key cryptography in which a publicly known string representing an individual or organization is used as a public key.
- The public string could include an email address, domain name, or a physical IP address.
- In most identity-based protocols it is not acceptable for a random string to be used as an identity, and therefore ID_I is typically calculated using a hash function.
- $ID_I = H(I)$ for a suitable hash function H , but in practice the information that is included in the hash may be much more extensive.

Identity Based Encryption (IBE)

Main idea The public key is an identity $ID \in \{0, 1\}^*$

A Key Generation Center **KGC** issues private keys for ID

An IBE scheme consists of 4 algorithms:

Setup Takes a security parameter ℓ and outputs system parameters **params** and **master-key**.

Encrypt Takes as inputs **params**, $ID \in \{0, 1\}^*$ and message M and outputs a ciphertext C .

Limitation

Suppose Alice and Bob are users of Id based system. Since the information needed to find Alice's public key is completely determined by Alice's ID and the master public key, it is not possible to revoke Alice's credentials and issue new credentials without either (a) changing Alice's ID (usually a phone number or an email address which will appear in a corporate directory); or (b) changing the master public key and re-issuing private keys to all users, including Bob.

Certificate revocation problem can be “avoided” using
 $ID = \text{bob@company.com}||\text{year}||\text{month}||\text{day}$

- The first implementation of identity-based signatures and an email-address based public-key infrastructure (PKI) was developed by Adi Shamir in 1984
- It allowed users to verify digital signatures using only public information such as the user's identifier.
- Under Shamir's scheme, a trusted third party would deliver the private key to the user after verification of the user's identity, with verification essentially the same as that required for issuing a certificate in a typical PKI.
- Shamir similarly proposed identity-based encryption, which appeared particularly attractive since there was no need to acquire an identity's public key prior to encryption.
- The first practical implementations were finally devised by Sakai in 2000 and Boneh and Franklin in 2001.
- These solutions were based on bilinear pairings.

Classification of ID schemes

- Identity-based schemes can be classified into three levels using a categorisation of Girault's .
- In the schemes at level 1 the public key of the entity is exactly the identity string ID_I .
- This means that once the description of a principal is obtained, its public key can be correctly calculated without recourse to any other information.
- No principal can be allowed to generate the corresponding private key.
- If this were the case then any entity could do the same; at the least this would mean that any entity could masquerade as any other.
- Therefore, in all identity-based schemes of this basic type the private key of each principal is calculated by a trusted third party.

Girault's level 2 protocol

- A number of level 1 key establishment protocols were proposed during the 1980s.
- Similar benefits can be achieved in key establishment protocols even when it is not possible to explicitly extract the public key from the identity.
- A value obtained from the trusted third party may be used in combination with the partner's identity to derive a key that can only be calculated by a principal with the correct private key. We call such a value an implicit certificate.
- This allows the private keys to be kept secret from the authority
- It can be regarded as a compromise between the basic identity based scheme and certificate-based schemes.

Limitations

- A certificate of a public key is a signature by a third party on certain information that includes the public key value.
- Any principal who receives such a certificate, including the owner of the private key, cannot use it to form another contradictory certificate (for example, one that shows a different public key).
- This is simply because only the third party can form new signatures.
- Certainly a malicious certification authority can generate its own private key and produce a certificate that shows that this private key belongs to any victim principal.
- This would allow the authority to masquerade as the principal.
- However, this certificate will contradict the real certificate of the principal.
- Because only the authority is able to produce a certificate the two contradictory certificates can be used to show that the authority has cheated.

- The problem with the early identity-based protocols, in which principals could choose their own private key, was that the principals could use their private information to form new implicit certificates.
- Since both the authority and the principal can form contradictory certificates it is impossible to tell which of them has cheated when two certificates appear.
- This situation is arguably very little better than when the authority has access to the private key, since it is able to masquerade as any principal without being caught.
- Finally in 1991 Girault solved this problem by defining self-certified public keys.
- These are keys that have an implicit certificate that can only be generated by the trusted authority.

Level	Properties	Example
1	Third party knows secret key	Okamoto
2	Third part cannot find private key. Principal can generate contradictory public key	Girault and Paillès
3	Only third party can generate valid public key	Girault

Okamoto's Protocol: A level 1 scheme

- The first published identity-based key agreement protocol.
- It uses a composite modulus $n = pq$ whose factorisation is known only to a trusted authority.
- The authority chooses values e and d as in the RSA algorithm, so that $ed = 1 \bmod \phi(n)$
- And an element g that is primitive in both the integers mod p and the integers mod q . The values g and e are made public.
- Before engaging in the key agreement protocol each user must register with the authority to obtain a private key.
- User I 's identification string, ID_I , is treated as an integer modulo n . The authority calculates the value $S_I = ID_I^{-d} \bmod n$ and distributes S_I securely to user I .

Shared information: Public modulus n and exponent e . Element g of high order in $\mathbb{Z}_n^*$.

Information known to A : s_A such that $s_A^e = ID_A^{-1} \bmod n$.

Information known to B : s_B such that $s_B^e = ID_B^{-1} \bmod n$.

A		B
$r_A \in_R \mathbb{Z}_n$		
$t_A = g^{r_A}$	$\xrightarrow{s_A t_A}$	$r_B \in_R \mathbb{Z}_n$
	$\xleftarrow{s_B t_B}$	$t_B = g^{r_B}$
$Z_{AB} = ((s_B t_B)^e ID_B)^{r_A}$		$Z_{AB} = ((s_A t_A)^e ID_A)^{r_B}$

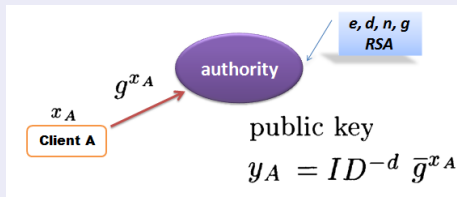
Protocol 5.27: Okamoto's identity-based protocol

- The shared secret is defined as $Z_{AB} = g^{er_A r_B}$.

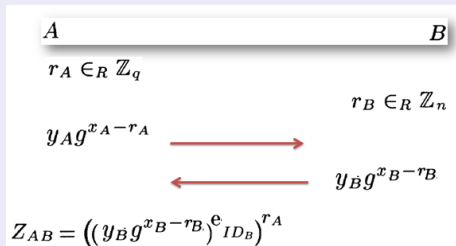
- On the assumption that it is necessary to know either S_A or S_B in order to find Z_{AB} , the scheme provides implicit key authentication.
- Similar to MTI protocol C(O) despite the different algebraic setting.
- Like that protocol, the Okamoto protocol is vulnerable to key compromise impersonation.
- Unknown key-share attacks seem to be prevented on the assumption that users cannot choose their own identities.
- Since compromise of the long-term private key of the trusted authority may allow the adversary to compute discrete logarithms in the group Z_n^* and thereby find old session keys. Therefore, forward secrecy is only provided if n is large enough that taking discrete logarithms in (at least one of) Z_p^* and Z_q^* is hard.

Girault and Pailles' protocol: A level 2 scheme

Registration



Key Agreement



Girault and Pailles' protocol

- Similar to Okamoto's scheme but lies in level 2 of Girault's classification
- The public key, Y_A , of A satisfies $Y_A^e ID_A = g^{-eX_A} \bmod n$, where g^{X_A} is given to the server but X_A is kept secret by A.
- If we rearrange this we find that $Y_A g^{X_A} = ID^{-d}$
- So that A can calculate Y_A herself when the same registration process as that in Okamoto's scheme is used.
- In the Girault and Pailles protocol A sends the message $Y_A g^{X_A - r_A}$ to B.
- But when we rearrange this message it becomes $ID^{-d} g^{-r_A}$, which is the same as the corresponding message sent in Protocol Okamoto scheme except for a change of sign.
- Similarly the shared secrets are identical in the two protocols.

Girault's Protocol: A level 3 scheme

- The algebraic setting for Girault's key agreement scheme using selfcertified keys is the same as that used for Okamoto's protocol.
- An RSA modulus n and key pair e, d are chosen by the server, together with an element g of high order in Z_n^* .
- When A registers to use the scheme she chooses her private key x_A and provides g^{x_A} to S , who calculates the self-certified public key $Y_A = (g^{x_A} - ID_A)^d \pmod{n}$.
- In order for the scheme to achieve level 3 it is essential that S is unable to find x_A .
- Saeednia has pointed out that a malicious server could choose n so that finding discrete logarithms is easy, and for this reason the size of n should preferably be 2048 bits.
- In addition S should provide a proof that n is the product of two safe primes

Girault's Protocol

Shared information: Public modulus n and exponent e . Element g of high order in $\mathbb{Z}_n^*$.

Information known to A : x_A with $y_A^e + ID_A = g^{x_A} \bmod n$.

Information known to B : x_B with $y_B^e + ID_B = g^{x_B} \bmod n$.

A		B
$r_A \in_R \mathbb{Z}_n$		
$t_A = g^{r_A}$	$\xrightarrow{t_A}$	$r_B \in_R \mathbb{Z}_n$
	$\xleftarrow{t_B}$	$t_B = g^{r_B}$
$Z_{AB} = t_B^{x_A} (y_B^e + ID_B)^{r_A}$		$Z_{AB} = t_A^{x_B} (y_A^e + ID_A)^{r_B}$

Protocol 5.32: Girault's identity-based protocol

The shared key is $Z_{AB} = g^{r_A x_B + r_B x_A}$.

- In Girault's original short paper he only mentioned the possibility of using the self-certified keys to produce an authenticated static shared key.
- If the public keys are already available this can be achieved with no message exchanges:
A calculates $(y_B^e + ID_B)^{x_A}$ in order to produce $Z_{AB} = g^{x_A x_B}$ and B calculates the analogous value.
- However, since the public key is simply a way to find an implicitly certified key for each party, any of the MTI protocols can be modified to provide a dynamic protocol. For example, Protocol 5.32 shows a version modified from MTI A(O), as suggested by Rueppel and van Oorschot.
- The shared key is $Z_{AB} = g^{r_A x_B + r_B x_A}$.
- Analogues of the other MTI protocols can be made similarly, replacing Y_A in the MTI original with $Y_A^e + ID_A$ in the Girault version, and similarly for Y_B .